AXSTRON GROUP · FINCRUSE-SETTLEMENT · BRANCH SETTLEMENT-ENGINE · V1 — USD CORRIDOR

# AxLink Settlement Blueprint

How value moves through the network, account by account and leg by leg. The customer deposits their own currency and withdraws their own currency; USD is the unit of account and USDT on Solana is the transport, and **neither is ever shown to them**. Every settlement is provable against a chain we do not control. This is the architecture as built on the `settlement-engine` branch, written so it can be implemented exactly.

---

## 01 What we are replacing

FIRST, WHAT THIS IS NOT

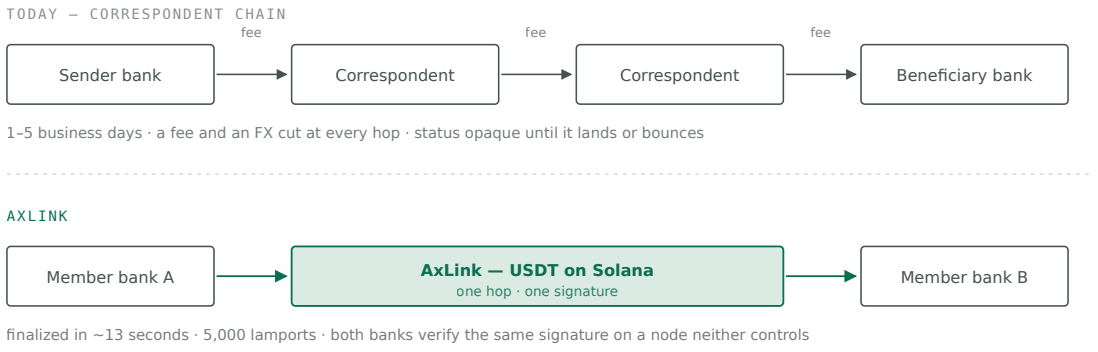
**This is back-office settlement infrastructure.** No customer sees it, and none of it is a product feature. A user deposits their own currency and withdraws their own currency; whether the value moved between our books on a correspondent chain or on Solana is invisible to them, and they have no way to tell. Nothing in this document should ever reach a customer-facing screen.

That invisibility is deliberate and it is worth something: **because the customer contract never references the rail, the rail can be replaced without touching the product.** A chain outage, a change of settlement asset, a regulator ruling a corridor out — each is an operations decision, not a migration.

So the case for this rail is an internal one — cost per transfer, how fast our own books can be trusted, and what we can hand an auditor. Not a story anyone tells a user.

A cross-border payment on correspondent banking is a chain of bilateral credits between banks that each hold accounts with the next. Nobody in the chain sees the whole path, each hop takes a fee and a cut of the spread, and the sender learns the outcome when it lands — or when it doesn't. Those costs land on *us*, and we either absorb them or price them into what the customer pays.

AxLink removes the intermediaries by giving both counterparties a position on one shared settlement asset and moving that asset directly. The proof matters to exactly three audiences, and a retail user is not among them: **a member bank on the other side of a transfer, an auditor, and us**.



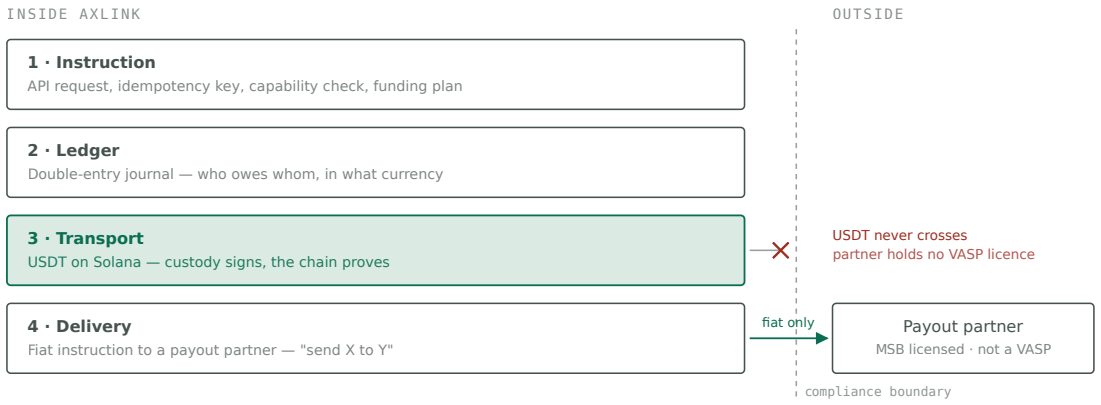

**The hops are the cost, and the cost is ours.** Each correspondent adds a fee, an FX cut, a day, and a counterparty to reconcile against. AxLink replaces the chain with one transfer of a shared asset. Solana's economics are what make the residual hop cheap enough to do per-transfer: 5,000 lamports is a fraction of a cent, and finality is 32 slots — roughly 13 seconds. A customer would experience either row identically; the difference shows up in our margin and our close.

WHY FINALITY IS ENFORCED ANYWAY

None of this reaches a customer, so the discipline has to be justified internally — and it is. **No transfer completes on a custodian's acknowledgement**, only on a finalized signature. Not because a user asked, but because it is the difference between books we can prove and books we merely believe: a settlement marked complete on an API response is a number nobody can check, including us, until the day someone has to.

## 02 The four planes

The system separates into four layers that fail independently and must never be confused with one another. Most of the bugs the current codebase carries come from collapsing two of them — treating a custody acknowledgement as settlement, or a balance update as an accounting entry.




**Only plane 4 crosses the boundary.** A payout partner receives an instruction to send fiat to a beneficiary and nothing else — no address, no token amount, no reference to a chain. The system enforces this in code: `resolveFundingPlan` refuses to construct a USDT plan for any partner not explicitly flagged `cryptoLicensed`.

### Why the separation is load-bearing

- **Ledger ≠ transport.** The journal records an obligation; the chain records that value moved. They are reconciled against each other daily, and a disagreement is a finding.
- **Transport ≠ custody.** Fireblocks signs. Solana settles. A Fireblocks transaction id proves a request was accepted, not that anything happened.
- **Delivery ≠ settlement.** A wire submitted to a converter is not a wire received by a partner. The leg stays `AWAITING_CONFIRMATION` until the partner says otherwise.

## 03 The currency model

Three currencies are in play at once and they are constantly confused. Naming them separately is what makes the rest of the system tractable.

| | |
|---|---|
| PRESENTATION | **The customer's own currency.** A Kenyan user deposits KES, sees a KES balance, withdraws KES. This is the only currency they ever encounter. |
| UNIT OF ACCOUNT | **USD.** Every internal position, limit and report is measured here. It is the pivot every corridor passes through. |
| SETTLEMENT ASSET | **USDT on Solana.** What actually backs the balances and moves between counterparties. Never shown, never sent to a partner. |

WHICH ENTITY HOLDS THE ASSET

The group is licensed differently in each market: FinCEN MSB in the USA, FINTRAC MSB in Canada, the UK entity, and a digital-asset regime in El Salvador. **Those are not interchangeable balance sheets.** A money-services licence permits moving fiat; it does not by itself permit holding a stablecoin position, and holding one in the wrong entity is a licensing problem rather than an accounting one.

So the USDT position belongs on the balance sheet of the entity licensed to hold digital assets, and the MSB entities hold a *fiat receivable from that entity* — never the token. Customers are unaffected either way; they see their own currency and never learn which entity settled behind it. But the books have to say it, and today they do not: `CUSTODY:*` accounts carry no entity dimension. See the gap noted below.

The settlement asset itself is one constant — `SETTLEMENT_ASSET` plus a mint address in a single table. Switching assets, or running a different one per region, stays a config change. **Cheap now, expensive once balances exist.**

The converter settles **USD, GBP and EUR** and nothing else. That single fact determines the shape of every corridor: those three are native, and everything else has to reach USD somehow. Which side needs the bridge is what separates an easy corridor from a hard one.

### Corridor classes

| CLASS | EXAMPLE | BRIDGE NEEDED | STATUS |
|---|---|---|---|
| **A — native** | USD in → USD out<br>GBP in → EUR out | None. The converter handles both ends. | BUILT |
| **B — local out** | USD in → KES out | Payout side only. We wire USD; the partner applies its own FX and delivers KES. | BUILT |
| **C — local in** | KES in → KES out | **Collection side.** Something must convert KES to USD and move it out of Kenya before the converter can see it. | UNSOLVED |

> THE ASYMMETRY — AND IT IS NOT A CODE PROBLEM
>
> Paying *out* in a local currency is easy: the partner already holds that currency and does the FX for us. Taking money *in* is the hard direction. The customer's shillings land in a Kenyan collection account, and the converter cannot accept shillings — so a **local FX and repatriation partner** has to convert KES to USD and wire it out. That is a banking relationship and a capital-controls question in each market, not something the settlement engine can solve. Kenya is workable; Nigeria is materially harder.
>
> **Name that partner per market before promising local deposits.** Until it exists, Class C corridors cannot run end to end no matter how correct the code is.

### Who carries the FX position

If the customer's balance is denominated in KES and the backing is USDT, **we are short KES against the customer and long USD** — a currency position nobody chose to take. It is small per customer and unbounded in aggregate, and it grows quietly with deposits.

Concretely: a user deposits KES 130,000 and is credited KES 128,700. We hold roughly 995 USDT. If the shilling strengthens from 130 to 120 against the

dollar, that USDT is worth KES 119,362 against a KES 128,700 liability — **short KES 9,338 on a single customer**, with no trade ever placed.

| OPTION | WHO BEARS THE RISK | COST |
|---|---|---|
| **Matched local float** hold KES against KES liabilities | Nobody — the position is flat | Working capital per market. Abandons the zero-pre-funding model. |
| **Priced spread** widen the deposit/withdrawal spread | Us, buffered by the margin | Uncompetitive pricing in volatile markets; the buffer fails exactly when it is needed. |
| **USD-denominated balance** display local at today's rate | The customer | Zero FX risk to us. The customer's local balance visibly moves day to day. |

> MAKE THIS A DECISION, NOT A DISCOVERY
>
> Whichever option you pick, the position has to be *measured*. Per-currency liability against USD-equivalent backing, reported daily. The journal already carries the data — `USER:*:KES` summed against custody, both in USD terms — but a report nobody reads is the same as no report. An FX book found during an audit is worse than one that was chosen.

## 04 Chart of accounts

Every movement has two sides. Before this existed, a withdrawal debited a customer and the money simply left the books — fees, FX spread, partner float and custody had no accounts to land in, so they were visible only as the arithmetic difference between a debit and a payout.

Codes are structured strings, built only through `AccountCode.*`. A free-form code would open a brand-new account and silently absorb money; an unrecognised prefix raises instead.

| CODE PATTERN | TYPE | HOLDS |
|---|---|---|
| `USER:<userId>:<CCY>` | Liability | What we owe a customer. Their app balance. |
| `BANK:<bankId>:<CCY>` | Liability | A member bank's position on the network. |
| `TREASURY:<CCY>` | Asset | Our own funds. |
| `CUSTODY:<vaultId>:<ASSET>` | Asset | USDT/USDT held in a named Fireblocks vault. |
| `COLLECTION:<rail>:<CCY>` | Asset `TRANSIENT` | Local currency collected at a rail, not yet repatriated. Class C corridors only. |
| `CONVERTER:<id>:<CCY>` | Asset `TRANSIENT` | In transit at B2BinPay. Should not persist across a close. |
| `FLOAT:<partner>:<CCY>` | Asset | Pre-funded balance sitting at a payout partner. |
| `FEE_INCOME:<CCY>` | Income | Our revenue. |
| `FX_PNL:<CCY>` | Income | Conversion clearing. Its net position *is* the spread. |
| `SUSPENSE:<CCY>` | Liability `TRANSIENT` | Debited from a customer, not yet delivered. |

GAP — ACCOUNTS CARRY NO ENTITY

Every code above is group-wide. In a multi-entity licensed group that is not enough: an auditor asks which *entity* holds a position, and a regulator asks whether that entity is permitted to. The fix is a fourth segment — `CUSTODY:sv:treasury:USDT`, `USER:us:<id>:USD` — plus `INTERCO:<from>:<to>:<CCY>` so positions between group entities are visible instead of netted invisibly into one balance sheet.

**Do this before there are balances.** Adding an entity dimension to a populated journal means restating history and deciding retroactively which entity held what — the kind of exercise that is straightforward on day one and a project on day two hundred.

SIGN CONVENTION

**Positive is a debit, negative is a credit**, in the line's own currency. Every posting sums to zero *per currency* — not in USD. A cross-currency posting that only balances after conversion is hiding an FX position, so conversions must name `FX_PNL` explicitly. That is what makes the converter's spread a number on a report instead of an unexplained shortfall.

## 05 Six primitives

Everything below is composed from these. Learn them once and every scenario reads the same way.

**`ProviderCapability`**

*what a partner can actually do*

Declares `settlementCurrencies` and `payoutCurrencies` *separately*, plus methods, countries, limits, and two flags that decide everything else: `cryptoLicensed` and `prefundedFloat`.

**`FundingPlan`**

*how this payout gets funded*

Resolved before any value moves. Names the settlement currency, the amount, who bears the FX, and — critically — `payoutPrecedesFunding`, which says whether the beneficiary waits for us.

**`SettlementLeg`**

*one step, written before it is taken*

The full plan is persisted in the same transaction as the domain record. Each leg carries a deterministic `externalKey` handed to the external system as its idempotency key, so a resumed leg reuses the same request instead of creating a second payment.

**`JournalEntry`**

*the books, append-only*

Balanced lines, asserted before write, keyed by a deterministic `postingKey`. Re-posting is a no-op — which is what makes the resume worker safe to run repeatedly. A correction is a reversing entry, never an edit.

**`SettlementProof`**

*evidence, not assertion*

Signature, slot, block time, mint, amount and fee actually paid — recorded only after an RPC node reports `finalized` *and* the on-chain movement matches what we intended.

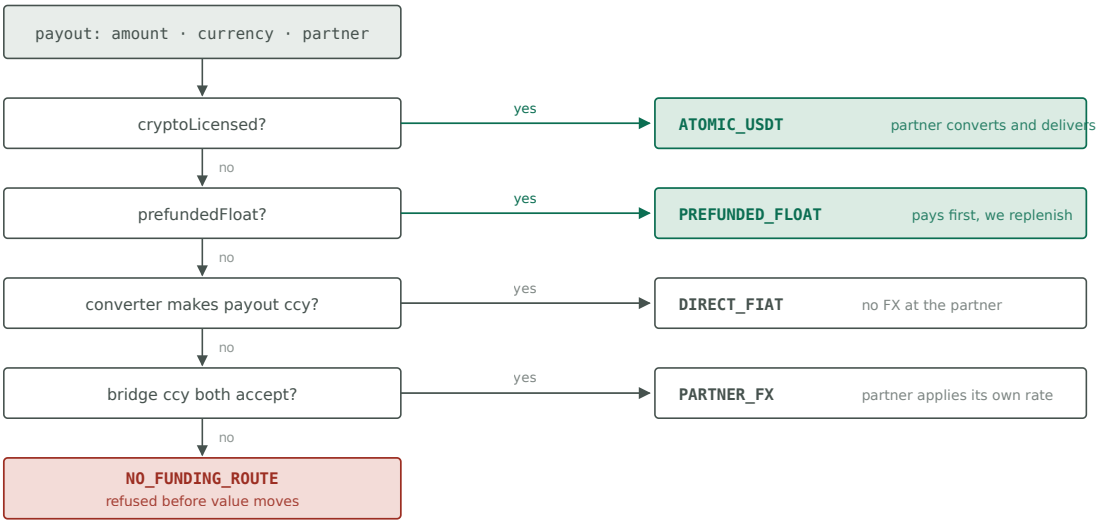
**`IdempotencyKey`**

*pay once*

Arbitrated by a unique constraint, not a read-then-write check — two concurrent requests both pass a read-then-write check. Same key with a different body is rejected, never served the cached response.

## 06 How a partner gets funded

This is the decision that makes corridors pluggable. It runs *before* any value moves, so a corridor with no viable route fails while the money is still ours — rather than after USDT has been moved to a converter that cannot produce the payout currency.




**Order matters.** Cheapest and least ambiguous first. The `cryptoLicensed` gate is not a preference — a partner whose capability lists USDT without the flag causes a hard refusal with an explanation, because a misconfiguration there is a regulatory exposure, not a routing inefficiency.

## Which mode a corridor lands in

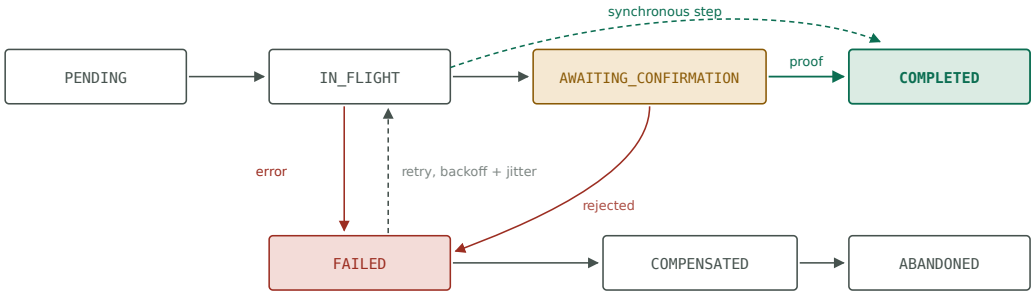
| MODE | PARTNER FUNDED IN | BENEFICIARY WAITS FOR OUR WIRE? | CAPIT |
|---|---|---|---|
| ATOMIC_USDT | USDT | No | None |
| PREFUNDED_FLOAT | USD (replenishment) | **No** — paid from float | Float per c |
| DIRECT_FIAT | Payout currency | Yes | None |
| PARTNER_FX | USD / EUR / GBP | Yes | None |

THE TRADE YOU HAVE TO MAKE

Our payout partners cannot hold crypto, which rules out `ATOMIC_USDT`. That leaves the two goals written throughout this system in direct tension: **zero pre-funded corridors** and **real-time execution**. You can have either, not both. Without float, the beneficiary cannot be paid until our fiat wire lands at the partner — banking rails, banking hours. `PREFUNDED_FLOAT` buys real-time for the recipient at the cost of working capital parked in each corridor, bounded by the float size. Both are implemented; it is a per-partner config flag, not a code change.

## 07 The leg state machine

A settlement is a sequence of legs, each of which either happened or did not. Persisting the plan before executing it is what makes a crash between the customer debit and the partner payout recoverable instead of permanent.




Terminal: COMPLETED · COMPENSATED · ABANDONED. Transitions are enforced, not documented — COMPLETED → IN_FLIGHT is how a system pays twice.

**Retries are bounded and then they stop.** A leg that exhausts `maxAttempts` stops retrying and appears in `GET /api/settlement/stuck`. That queue must be empty at the start of each day: every row in it is customer money in an indeterminate state, and no amount of retrying will resolve it.

## 08 Scenarios

Worked end to end, with the actual postings. Figures are illustrative but internally consistent — the books balance in every one, and you can check them.

### Add funds — $10,000 into an Appra account HAPPY PATH

A customer sends $10,000 by bank transfer. Our deposit fee is 25 bp ($25). B2BinPay's conversion spread costs 15 bp ($15), embedded in the USDT we receive: we get 9,985 USDT for $10,000.

| LEG 0 | LEG 1 | LEG 2 | LEG 3 |
|---|---|---|---|
| CONVERT_FIAT_TO_CRYPTO | CUSTODY_TRANSFER | ONCHAIN_SETTLE | LEDGER_CREDIT |
| ● awaits converter | ● awaits broadcast | ● finalized | ● spendable |

THIS IS WHAT WAS BROKEN

The previous implementation called `createDeposit` and then, in the same function, immediately requested a USDT payout to the user's vault — **paying out USDT that had not been minted yet**, because the conversion had not happened. Leg 1 does not start until leg 0 is confirmed. That ordering is the entire reason legs exist.

`BankDeposit:<id>:fiat-received`

on rail confirmation — the customer's claim exists from here

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| CONVERTER:b2binpay:USD | 10,000.00 | — | USD |
| USER:u_812:USD | — | 9,975.00 | USD |
| FEE_INCOME:USD | — | 25.00 | USD |
| USD residual | 10,000.00 | 10,000.00 | 0.00 ✓ |

`BankDeposit:<id>:conversion`

after the signature is finalized — FX_PNL clears both currency legs

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| CONVERTER:b2binpay:USD | — | 10,000.00 | USD |
| FX_PNL:USD | 10,000.00 | — | USD |
| CUSTODY:vault_812:USDT | 9,985.00 | — | USDT |
| FX_PNL:USDT | — | 9,985.00 | USDT |
| Per-currency residual | USD 0.00 ✓ | USDT 0.00 ✓ | balanced |

CHECK THE BOOKS

Assets 9,985.00 USDT. Liabilities 9,975.00 to the customer. Income: +25.00 fee, −15.00 FX loss (the converter's spread, now sitting visibly in `FX_PNL`) = +10.00. Assets minus liabilities equals retained earnings: 9,985 − 9,975 = 10. **The customer's balance is over-collateralised by $10, and you can see exactly why.** Under the old single-entry ledger this $15 spread had nowhere to land and would have appeared as an unexplained shortfall.

| | |
|---|---|
| CUSTOMER SEES | Deposit pending → **$9,975.00 available** once leg 3 posts. Fee shown at $25.00. |
| ELAPSED | Bank rail: hours to a day. Conversion: minutes. On-chain leg: **~13s**. |
| IDEMPOTENCY | `deposit` scope, key `<rail>:<railReference>` — the same bank credit can never be booked twice. |

## Withdrawal — $5,000, JIT wire to the partner

NOT REAL-TIME FOR THE RECIPIENT

Customer withdraws $5,000. Our fee is 50 bp ($25), so the beneficiary is owed $4,975. Converting back costs 15 bp, so we move 4,982.50 USDT to produce $4,975.

| LEG 0 | LEG 1 | LEG 2 | LEG 3 |
|---|---|---|---|
| CUSTODY_TRANSFER | ONCHAIN_SETTLE | CONVERT_CRYPTO_TO_FIAT | PARTNER_PAYOUT |
| ● vault → treasury | ● finalized | ● USD at converter | ● awaits partner |

`Withdrawal:<id>:debit`

balance check and debit in one transaction — checking outside it lets two concurrent withdrawals both pass

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| USER:u_812:USD | 5,000.00 | — | USD |
| SUSPENSE:USD | — | 4,975.00 | USD |
| FEE_INCOME:USD | — | 25.00 | USD |
| USD residual | 5,000.00 | 5,000.00 | 0.00 ✓ |

`Withdrawal:<id>:custody`

leg 0 — USDT collected into treasury, proven by leg 1 before anything else runs

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| CUSTODY:treasury:USDT | 4,982.50 | — | USDT |
| CUSTODY:vault_812:USDT | — | 4,982.50 | USDT |
| USDT residual | 4,982.50 | 4,982.50 | 0.00 ✓ |

`Withdrawal:<id>:conversion`

leg 2 — USDT out, USD at the converter, spread visible in FX_PNL

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| CUSTODY:treasury:USDT | — | 4,982.50 | USDT |
| FX_PNL:USDT | 4,982.50 | — | USDT |
| CONVERTER:b2binpay:USD | 4,975.00 | — | USD |
| FX_PNL:USD | — | 4,975.00 | USD |
| Per-currency residual | USD 0.00 ✓ | USDT 0.00 ✓ | balanced |

`Withdrawal:<id>:delivered` only on the partner's confirmation — suspense returns to zero

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| SUSPENSE:USD | 4,975.00 | — | USD |
| CONVERTER:b2binpay:USD | — | 4,975.00 | USD |
| USD residual · SUSPENSE now 0.00 | 4,975.00 | 4,975.00 | 0.00 ✓ |

WHERE "REAL-TIME" STOPS

Legs 0–2 complete in seconds. Leg 3 is a bank wire: B2BinPay debits our wallet immediately, but the money reaches the partner on banking rails, in banking hours. It sits in `AWAITING_CONFIRMATION` — **never `COMPLETED` on submission**. A system that marks it complete when the API returns 200 is a system that reports settled payments still sitting in a correspondent's queue.

## Withdrawal — $5,000 against a pre-funded float

REAL-TIME FOR THE RECIPIENT

Same customer, same amount, one flag different: `capability.prefundedFloat = true`. The partner already holds our money, so they pay the beneficiary now and our wire replenishes them afterwards. The leg order inverts — payout first, funding second.

| LEG 0 | LEG 1 | LEG 2 | LEG 3 |
|---|---|---|---|
| PARTNER_PAYOUT | CUSTODY_TRANSFER | ONCHAIN_SETTLE | CONVERT_CRYPTO_TO_FIAT |
| ● paid from float | ● replenishment | ● finalized | ● wire to float |

`Withdrawal:<id>:debit`

the beneficiary is paid out of FLOAT, so our asset at the partner falls immediately

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| USER:u_812:USD | 5,000.00 | — | USD |
| FLOAT:partner_x:USD | — | 4,975.00 | USD |
| FEE_INCOME:USD | — | 25.00 | USD |
| USD residual · no suspense used | 5,000.00 | 5,000.00 | 0.00 ✓ |

`Withdrawal:<id>:replenish` off the critical path — the customer is already paid

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| FLOAT:partner_x:USD | 4,975.00 | — | USD |
| CONVERTER:b2binpay:USD | — | 4,975.00 | USD |
| USD residual · float restored | 4,975.00 | 4,975.00 | 0.00 ✓ |

| | |
|---|---|
| RECIPIENT | **Seconds.** This is the only configuration where "real-time" describes the beneficiary's experience rather than our internal transport. |
| CAPITAL AT RISK | The float balance, and nothing more. Size it to expected daily volume, not to peak. |
| IF REPLENISHMENT FAILS | The customer's payout is **not** failed — they already have the money. The error is logged loudly and the float is short until it clears. Monitor `FLOAT:*` balances, not just the payout queue. |

### The RPC node is unreachable mid-settlement FAILURE

USDT has been moved at the custodian. We go to verify the signature and the Solana node times out. **Three outcomes must stay distinct here, and conflating them loses money.**

| WHAT WE OBSERVE | WHAT IT MEANS | WHAT THE SYSTEM DOES |
|---|---|---|
| Signature finalized, no error | Settled | Record `SettlementProof`, leg → `COMPLETED` |
| Signature landed, on-chain error | **Did not settle** | `SettlementNotFinalError`, proof row records the failure, leg → `FAILED` |
| Cannot reach any node | **Nothing at all** | `PROOF_UNAVAILABLE`, `retryable`, leg stays `AWAITING_CONFIRMATION` |

> NEVER TREAT A TIMEOUT AS A FAILURE
>
> A timeout says the call did not *answer* in time, not that it did not *happen*. Reversing a transfer because an RPC node was slow reverses a transfer that may well have settled. The leg holds, the resume worker polls, and if it never resolves it surfaces in `/api/settlement/stuck` for a person.

The practical version of this failure is mundane: `SOLANA_RPC_URL` left pointing at the public mainnet endpoint. It rate-limits under load, verification starts failing, and because the engine correctly refuses to complete unproven transfers, the visible symptom is settlements piling up. Use a dedicated node.

### The partner rejects the payout FAILURE

Beneficiary account closed. The partner returns the funds. The customer was debited $5,000 and $4,975 is sitting in suspense; the fee has already been recognised.

`Withdrawal:<id>:debit:reversal`

the journal is append-only — a correction is a new, linked entry, never an edit

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| USER:u_812:USD | — | 5,000.00 | USD |
| SUSPENSE:USD | 4,975.00 | — | USD |
| FEE_INCOME:USD | 25.00 | — | USD |
| USD residual · suspense clears, fee un-earned | 5,000.00 | 5,000.00 | 0.00 ✓ |

Both entries stay on the customer's statement, linked by `reversesId` — the debit and the reversal, not a vanished transaction. The USDT already converted to USD remains at the converter as treasury; the round-trip conversion cost is a real loss and lands in `FX_PNL` rather than being quietly netted out of the customer's balance.

WHAT THE OLD CODE DID

Credited the gross amount back and moved on. The custody leg was never reversed, the fee was silently retained with no entry, and the conversion cost had nowhere to land — three separate holes in the books from one failed payout.

### The client retries and sends the request twice FAILURE

A mobile client times out and resubmits. Two identical withdrawal requests arrive concurrently, milliseconds apart, on two application instances.

```
// Both callers race a unique constraint. Exactly one wins the insert.
prisma.idempotencyKey.create({ data: { scope: 'withdrawal', key, requestHash, sta

// Winner  → runs the work, stores the response, returns EXECUTED
// Loser   → P2002 violation → routed to the replay path
//   same requestHash + COMPLETED  → REPLAYED, returns the stored response
//   same requestHash + IN_FLIGHT  → 409 REQUEST_IN_FLIGHT
//   different requestHash         → 409 IDEMPOTENCY_CONFLICT — never the cached
```

| | |
|---|---|
| WHY NOT A CHECK | The old `checkIdempotency` read for an existing row and then wrote. Two concurrent callers both read nothing, both passed, and **both settled** — and there was no unique constraint behind it to catch them. |
| STALE IN-FLIGHT | A row held longer than 5 minutes means the process holding it is gone. A retry takes it over. Five minutes comfortably exceeds every partner timeout we set. |
| CHANGED BODY | Rejected outright. Reusing a reference with different parameters is a client bug, and serving the first payment's response to the second request would hide it. |

### KES in, KES out — what the customer never sees DEPOSIT LEG NOT BUILT

A Nairobi customer pays KES 130,000 into M-Pesa and later withdraws KES 100,000 to a different number. They see shillings at both ends and nothing

else. Underneath, the value crosses two FX boundaries and one chain.

DESIGNED HERE, NOT YET IMPLEMENTED

The withdrawal half runs on the code as built. The deposit half needs the **KES → USD repatriation partner** from the currency model — the converter cannot accept shillings, so something has to convert and wire them out. `addFunds` is currently USD-only and rejects anything else rather than pretending.

DEPOSIT — KES 130,000

| LEG 0 | LEG 1 | LEG 2 | LEG 3 |
|---|---|---|---|
| COLLECT_LOCAL | REPATRIATE_FX | CONVERT_FIAT_TO_CRYPTO | ONCHAIN_SETTLE |
| ● M-Pesa confirms | ● partner needed | ● USD → USDT | ● finalized |

LEG 4

LEDGER_CREDIT

● spendable

`BankDeposit:<id>:fiat-received`

M-Pesa confirms · 1% deposit fee, charged in the customer's own currency

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| COLLECTION:daraja:KES | 130,000.00 | — | KES |
| USER:u_441:KES | — | 128,700.00 | KES |
| FEE_INCOME:KES | — | 1,300.00 | KES |
| KES residual | 130,000.00 | 130,000.00 | 0.00 ✓ |

`BankDeposit:<id>:repatriation`

the missing leg — local partner converts at 130.5 and wires USD out of Kenya

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| COLLECTION:daraja:KES | — | 130,000.00 | KES |
| FX_PNL:KES | 130,000.00 | — | KES |
| CONVERTER:b2binpay:USD | 996.17 | — | USD |
| FX_PNL:USD | — | 996.17 | USD |
| Per-currency residual | KES 0.00 ✓ | USD 0.00 ✓ | balanced |

`BankDeposit:<id>:conversion`

USD → USDT at 15 bp · the customer's shillings are now on a chain

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| CONVERTER:b2binpay:USD | — | 996.17 | USD |
| FX_PNL:USD | 996.17 | — | USD |
| CUSTODY:vault_441:USDT | 994.68 | — | USDT |
| FX_PNL:USDT | — | 994.68 | USDT |
| Per-currency residual | USD 0.00 ✓ | USDT 0.00 ✓ | balanced |

> THE POSITION THIS OPENS
>
> We now owe the customer **KES 128,700** and hold **994.68 USDT**. At 130 that backing is KES 129,308 — covered by 608 shillings. At 120 it is KES 119,362, and we are short KES 9,338. The customer is unaffected either way; the exposure is entirely ours, and it is the reason the currency-model decision cannot be deferred.

WITHDRAWAL — KES 100,000

This half runs today. The partner is funded in USD and applies its own rate — `PARTNER_FX`, `fxBearer: 'PARTNER'` — so the delivered figure is confirmed by them, not promised by us.

`Withdrawal:<id>:debit` 1% fee · suspense opens in KES, the currency we owe

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| USER:u_441:KES | 100,000.00 | — | KES |
| SUSPENSE:KES | — | 99,000.00 | KES |
| FEE_INCOME:KES | — | 1,000.00 | KES |
| KES residual | 100,000.00 | 100,000.00 | 0.00 ✓ |

`Withdrawal:<id>:conversion`

765.63 USDT → USD 764.48 · sized from the partner's indicative 129.5

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| CUSTODY:treasury:USDT | — | 765.63 | USDT |
| FX_PNL:USDT | 765.63 | — | USDT |
| CONVERTER:b2binpay:USD | 764.48 | — | USD |
| FX_PNL:USD | — | 764.48 | USD |
| Per-currency residual | USD 0.00 ✓ | USDT 0.00 ✓ | balanced |

`Withdrawal:<id>:delivered`

partner confirms KES 99,000 paid to the beneficiary — suspense clears, FX_PNL bridges the two currencies

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| SUSPENSE:KES | 99,000.00 | — | KES |
| FX_PNL:KES | — | 99,000.00 | KES |
| FX_PNL:USD | 764.48 | — | USD |
| CONVERTER:b2binpay:USD | — | 764.48 | USD |
| Per-currency residual · SUSPENSE:KES now 0.00 | KES 0.00 ✓ | USD 0.00 ✓ | balanced |

CUSTOMER SEES

Paid KES 130,000 by M-Pesa, balance KES 128,700, withdrew KES 100,000, beneficiary received KES 99,000. **No dollars, no token,**

| | |
|---|---|
| | **no chain.** |
| CROSSED UNDERNEATH | KES → USD → USDT → Solana → USD → KES. Two FX boundaries, one signature. |
| WHERE MARGIN LANDS | `FEE_INCOME:KES` is our revenue. `FX_PNL` accumulates every spread we paid — repatriation, both conversions, and the partner's rate. Read per corridor: this is where a bad partner rate becomes visible rather than absorbed. |

### Adding KES via M-Pesa — no core changes EXTENSION

The scenario above shows the money moving. This shows the code that makes it possible. Safaricom is an MNO with a mobile-money licence, not a VASP: it cannot receive crypto, and the converter cannot produce KES. Under the old `atomicJit: boolean` this corridor had no representation at all and threw — **which is why the entire Africa book did not work**.

```
// The whole integration. No changes to the orchestrator, the ledger, or the engi
readonly capability = {
  settlementCurrencies: ['USD'],          // fund us in dollars
  payoutCurrencies:     ['KES', 'TZS', 'UGX'],
  methods:              ['mobile_money'],
  countries:            ['KE', 'TZ', 'UG'],
  partnerAppliesFx:     true,             // they convert USD → KES at their rat
  cryptoLicensed:       false,            // hard block on any USDT plan
  prefundedFloat:       false,            // flip to true when a float is funded
};
```

The resolver reaches step 4 and returns `PARTNER_FX`: fund in USD, size the wire from an indicative USD/KES rate, and mark `fxBearer: 'PARTNER'` so the delivered amount is understood to be confirmed by Safaricom rather than promised by us.

> THE POINT OF THE ABSTRACTION
>
> A new corridor is a capability declaration and a provider class. It touches no shared code path, so it cannot break USD. That is the test of whether "plug in any payout partner" is real or just an interface with the word `interface` in front of it.

### Bank to bank — the MT103 replacement NETWORK

Two member banks, each with a vault and a position on the network. Bank A instructs a transfer to Bank B against its own reference, which is the idempotency key.

`InterbankTransfer:<id>:settle` USD 250,000 — positions move, then the chain proves it

| ACCOUNT | DEBIT | CREDIT | CCY |
|---|---|---|---|
| BANK:axlk_ke_001:USD | 250,000.00 | — | USD |
| BANK:axlk_gb_004:USD | — | 249,875.00 | USD |
| FEE_INCOME:USD | — | 125.00 | USD |
| USD residual · 5 bp network fee | 250,000.00 | 250,000.00 | 0.00 ✓ |

**PROOF**

Both banks fetch `GET /api/settlement/proof/InterbankTransfer/<id>`. It is **unauthenticated on purpose** — a proof only we can read proves nothing. The response carries the signature, slot and an explorer link they can check on a node neither of us runs.

**NOTIFICATION**

Written to the outbox in the same transaction as the settlement, HMAC-signed, delivered at-least-once with backoff. Receivers dedupe on `trackingId`. Previously this was a floating `.then()` — a partner returning 500 lost the notification permanently, on a network where the webhook *is* how the receiving bank learns it has money.

**LIMITS**

Daily cap per bank and per-corridor caps, checked before the transfer is created.

## 09 Invariants

These are the properties the system is built to preserve. Each one has a check that enforces it and a place it surfaces when it breaks. An invariant with no check is a wish.

**I-1** **Every posting balances to zero, per currency.**

Asserted in `assertBalanced` before write. A break in the daily trial balance means a write bypassed the journal — reported CRITICAL, because it invalidates every other number.

**I-2** **No transfer completes without a finalized signature.**

Reconciliation flags any `COMPLETED` withdrawal with no `SettlementProof`. Proving against a stub custody id is refused outright.

**I-3** **Suspense is zero at close of day.**

A non-zero balance is a customer debited and not delivered. This is an incident, not a reconciling item.

**I-4** **`Account.balance` equals the journal.**

Both written in one transaction. The projection-drift check is the tripwire for a legacy `debit` / `credit` call that skipped the journal.

**I-5** **One idempotency key pays once.**

Enforced by a unique constraint, not application logic.

**I-6** **No crypto reaches an unlicensed partner.**

Enforced in the funding resolver, which fails closed when `cryptoLicensed` is merely unset rather than assuming.

**I-7** **Our books agree with custody, and custody agrees with the chain.**

Two separate checks — that is what tells you whether a discrepancy is ours or the custodian's.

**I-8** **`/settlement/stuck` is empty each morning.**

Every row is money in an indeterminate state that retrying will not fix.

**I-9** **The FX position is measured daily, per currency.**

Local-currency liabilities against USD-equivalent backing. A position that is small today and unmeasured is an FX book found during an audit rather than one that was chosen.

## 10 Trust model

Worth being explicit about, because "trustless" is usually overclaimed. Here is what each party must actually take on faith, and what they can check for themselves.

| PARTY | MUST TRUST | CAN VERIFY INDEPENDENTLY |
|---|---|---|
| Customer | That we hold their balance and will honour it | Their statement, in their own currency. **They see no rail, no asset and no proof** — and would not know if the rail changed. |
| Member bank | Our FX rate and fee at quote time | **That settlement happened** — signature, slot, amount, on any node |
| Payout partner | That our wire will arrive | Their own statement against our `ProviderAttempt` log |
| Regulator | Nothing structural | Trial balance, journal, proofs, reconciliation history |
| Us | Custodian signs what we asked; converter honours its rate | Both — against the chain, and against the journal |

The honest summary: **settlement is verifiable, custody is not**. We still trust Fireblocks to sign correctly and B2BinPay to convert at the quoted rate. What we no longer do is trust either of them about whether value actually moved — that question is answered by a chain, and the answer is recorded whether it agrees with them or not.

## 11 Rollout

Genuinely sequential — each phase depends on the one before it being true in production, not merely deployed.

PHASE 1 · BEFORE ANY LIVE MONEY

### Baseline the schema and prove the books

- Run `scripts/create-baseline-migration.sh` and commit `prisma/migrations/`. Until this exists, `prisma migrate deploy` silently does nothing and a deploy succeeds against an unmigrated database.
- Point `SOLANA_RPC_URL` at a dedicated node.
- Run a full reconciliation on an empty database. It must come back CLEAN.
- **Decide the entity dimension now.** Which entity holds the USDT position, and which hold fiat receivables against it. Retrofitting this onto a populated journal means restating history.

PHASE 2 · SANDBOX, NO CUSTOMER FUNDS

### Walk the USD corridor end to end

- Deposit, then withdrawal, against sandbox credentials. Check the trial balance after each.
- Kill the process mid-settlement and confirm the resume worker completes it.
- Submit a duplicate withdrawal and confirm exactly one payout.
- Point at a bad RPC URL and confirm settlements hold rather than fail.

PHASE 3 · LIVE, CAPPED

### USD only, one partner, low limits

- JIT wire mode. Not real-time for the recipient, and the product copy must say so.
- Daily reconciliation, reviewed by a person. Suspense checked at close.
- Cap per transaction and per day until a month of clean reconciliations.

PHASE 4 · THE REAL-TIME DECISION

### Fund a float, if the working capital case holds

- Size the float to expected daily volume. Flip `prefundedFloat` on that partner alone.
- Monitor `FLOAT:*` as a first-class balance — a failed replenishment is now a treasury exposure, not a customer-facing failure.

PHASE 5 · EXPANSION

### KES, then NGN

- **Withdrawals first.** Paying out in local currency is Class B and works today — a capability declaration plus a provider class, no core changes. If a corridor needs a core change, the abstraction is wrong and should be fixed rather than worked around.
- **Local deposits are gated on a repatriation partner.** Class C cannot run until someone is contracted to convert KES to USD and wire it out. Name them, and settle the capital-controls question, before the feature is promised.
- Settle the currency-model decision before the first local deposit lands, not after. Once local-denominated balances exist, the FX position exists.
- Watch `FX_PNL` per corridor: under `PARTNER_FX` the partner sets the rate, so this is where a bad partner rate becomes visible instead of absorbed.

## 12 Not yet

Deliberately out of v1, recorded so they are decisions rather than oversights.

### Netting

Every transfer currently settles gross — one on-chain hop per payment. At 5,000 lamports that is survivable, but it forfeits the main structural advantage of running your own network: a thousand transfers between two member banks in a window should net to **one** settlement leg, and per-transfer confirmation latency should come off the critical path. The schema was drafted and then removed rather than shipping unused tables. It becomes worthwhile at the point where two member banks have real bilateral volume, which is a Phase 5 conversation.

### Migrating the legacy ledger helpers

`debit` and `credit` in `ledger.service.ts` still mutate balances without posting. The USD corridor does not use them; older services do. The projection-drift check catches any that remain, which is the safe order — detect first, migrate second.

### Decimal handling in the older services

`Decimal(20,8)` columns are still read into JavaScript `number` across the pre-existing code, reintroducing exactly the float error the column type was chosen to avoid. The settlement engine uses `Prisma.Decimal` throughout and never converts.

## 13 Reference

### Operations surface

All under `/api/settlement`, gated by `OPS_API_KEY` — with one deliberate exception.

| ENDPOINT | ANSWERS |
| --- | --- |
| `GET /proof/:type/:id` | **Public.** Signature, slot, explorer link. |
| `GET /books/trial-balance` | Do the books balance, per currency? |
| `GET /books/account/:code` | Statement for one account. |
| `GET /legs/:type/:id` | Where is this settlement, leg by leg? |
| `GET /stuck` | What needs a human? Must be empty each morning. |
| `POST /advance/:type/:id` | Nudge one settlement forward by hand. |
| `POST /withdrawals/:id/confirm` | Partner confirmed the wire. The only path that closes a withdrawal. |
| `POST /reconcile` | Run reconciliation. Daily. |
| `GET /providers/health` | Circuit state and payout success rate — over payouts, not quotes. |
| `GET /outbox` | Undelivered webhooks. |
| `GET /infra` | Can we reach a Solana node? Warns if the RPC is the public endpoint. |

## Where each piece lives

| CONCERN | MODULE |
|---|---|
| Chart of accounts | `core/settlement/accounts.ts` |
| Journal, trial balance | `core/settlement/journal.service.ts` |
| Leg state machine | `core/settlement/leg.service.ts` |
| USD deposit and withdrawal | `core/settlement/usd-corridor.service.ts` |
| Crash recovery | `core/settlement/resume.worker.ts` |
| On-chain proof | `core/settlement/proof.service.ts` |
| Solana RPC | `providers/live/solana.ts` |
| Reconciliation | `core/settlement/reconciliation.service.ts` |
| Capability and funding plan | `core/provider/capability.ts` |
| Circuit breaker, attempt log | `core/provider/` |
| Idempotency | `core/idempotency.service.ts` |
| Outbox | `core/outbox.service.ts` |

---

Architecture as implemented on the `settlement-engine` branch of `AXStronltd/Fincruse-settlement`. Figures in the scenarios are illustrative and internally consistent; fee and spread assumptions are ours to set. Solana timing and fee figures are typical mainnet values, not guarantees. Companion documents in the repo: `docs/GAP_ANALYSIS.md` for what was wrong and why, `docs/DEPLOY.md` for getting it running.